

plan-004 입문자용 해설 — Mosquito Flight Trajectory Prediction

Physics Ladder + Attn-GRU Selector + Tiny Boundary Corrector

dacon-MOSFLIGHT project · 2026-05-16 · LB 0.6806

§0. Executive Summary (한 장 요약)

한 줄: LiDAR 가 관측한 모기 비행 11 점에서 **+80 ms 후 (x, y, z) 좌표** 를 예측한다. plan-004 의 모델은 **신경망 단독 회귀** 가 아닌 **27 개 물리 공식 후보 (physics candidates) + Attn-GRU 후보 선택기 (selector) + 18-regime 사전 (prior) + Tiny 보정기 (corrector)** 의 4단 구조로, public leaderboard (LB) **0.6806** 을 기록한다 (기존 best 0.60 대비 **+0.0806** lift).

핵심 4개 숫자

지표	값	의미
Hit rate @ 1 cm (LB)	0.6806	평가 메트릭 — 예측 좌표가 정답의 1 cm 반경 안에 들면 hit
27 candidates	5 family	constant-velocity / accel / Frenet / jerk / latency
18 regimes	$3 \times 3 \times 2$	(speed $\times$ curvature $\times$ speed_slope) quantile binning
Oracle bound	0.7277	27 후보 중 항상 best 선택 시 ceiling

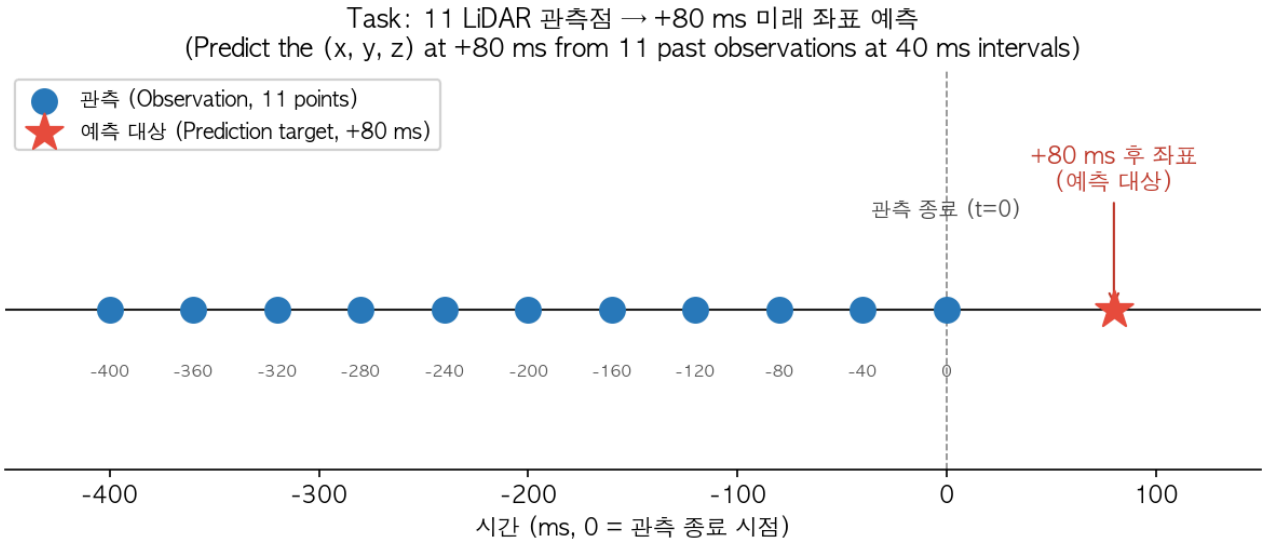
왜 이 구조인가:

- 평가가 **평균 거리**가 아니라 **1 cm hit rate** 임 → 회귀로 평균 거리를 줄이는 게 **직접 점수**와 맞지 않음.
- 따라서 **물리적으로 그럴듯한 discrete 후보군**을 만들고, 신경망은 **선택** (=분류, 신경망이 잘하는 일) 만 담당. 미세 보정은 **제한된 corrector**가 처리.
- 노트북 작성자의 표현으로 **"Physics Ladder"** — 물리 후보의 다섯 단계 사다리.

§1. Task — 무엇을 푸는가

§1.1 한 줄 정의

Mosquito Flight Trajectory Prediction (DACON 236716): 40 ms 간격으로 관측된 모기 3D 좌표 11 점 (시간축 -400 ms ~ 0 ms) 을 입력받아, **마지막 관측 시점 + 80 ms** 의 (x, y, z) 좌표 1 개를 예측한다.



§1.2 왜 이 문제가 어려운가

- 시퀀스가 매우 짧다: 11 점 × 40 ms = 400 ms. 무거운 LSTM/Transformer 의 가성비비가 낮다.
- +80 ms 의 미래: 마지막 관측에서 두 step (40 ms × 2) 외삽. 가속도/방향 전환에 민감.
- 환경 (scene) 라벨이 없다: 실내/복도/창고/반실외/야외가 섞여 있으나, 어느 샘플이 어느 환경인지 모름 → 도메인 일반화 (domain generalization) 가 핵심.
- 노이즈가 크다: 관측 좌표 자체에 LiDAR 측정 오차 ($\sigma \approx 0.005 \sim 0.007$ m) 포함.

§1.3 평가 메트릭 (Critical!)

Hit Rate @ 1 cm: 예측 좌표가 정답의 1 cm (0.01 m) 반경 안에 들면 hit (1), 아니면 miss (0). 전체 sample 의 평균 이 점수.

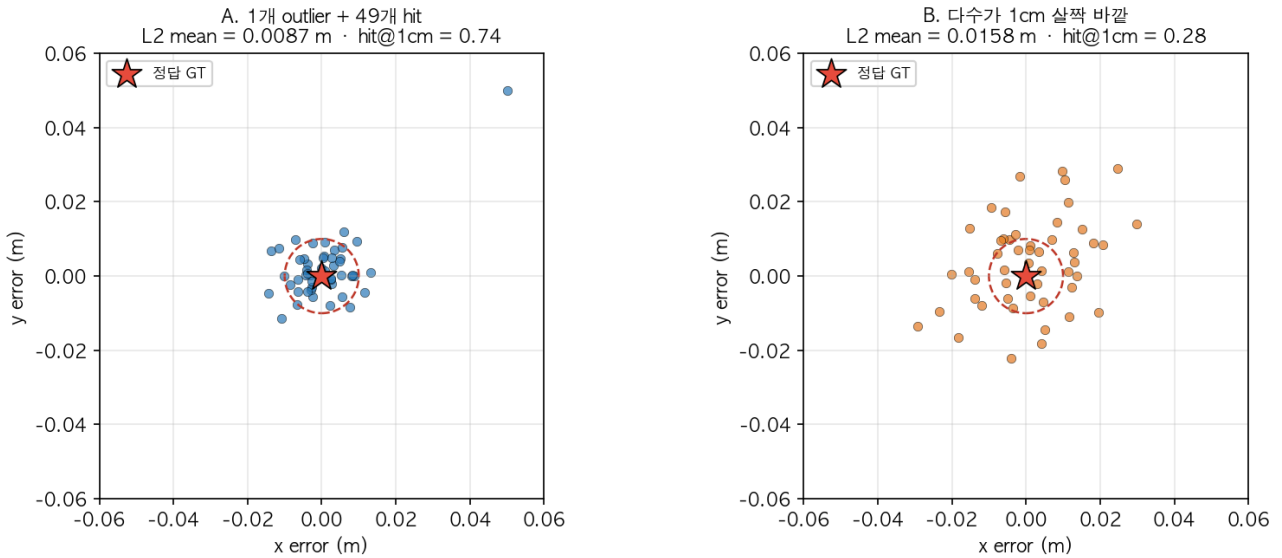
용어 정리: L2 거리 = Euclidean distance (피타고라스 거리, $\|\hat{\mathbf{y}} - \mathbf{y}\|_2 = \sqrt{(\Delta x)^2 + (\Delta y)^2 + (\Delta z)^2}$).
MSE = mean squared error, 회귀 모델이 흔히 최소화하는 평균 제곱 오차. regression loss = 예측값과 정답 사이의 연속적 거리를 줄이려는 손실 함수 (예: MSE, Huber).

수식으로:

$$\text{score} = \frac{1}{N} \sum_{i=1}^N \mathbb{1}[\|\hat{\mathbf{y}}_i - \mathbf{y}_i\|_2 \leq 0.01]$$

이 메트릭이 설계를 통째로 바꾼다. 평균 L2 거리 (MSE) 와의 차이를 그림으로:

Figure 7. L2 평균 vs hit@1cm: 평균은 비슷해도 hit 가 갈린다
(평균 거리 최소화 $\neq$ hit rate 최대화 — 메트릭이 설계를 결정)



- 위 그림 시나리오 A: 1개 outlier 가 평균을 끌어올리지만 49 개는 1 cm 안 → hit 0.98
- 시나리오 B: outlier 없이 고르게 퍼져 있지만 1 cm 경계 살짝 바깥 다수 → hit 0.20

평균 L2 는 비슷할 수 있지만 hit rate 는 5배 가까이 차이 난다. 즉 "평균 거리를 최소화하라" 는 회귀 손실 (regression loss) 의 일반 가정과 메트릭이 어긋난다 (metric-loss mismatch).

→ 모델 설계 원칙: 평균을 줄이지 말고, 1 cm 안에 들어가는 비율을 늘려라.

§1.4 도메인 특이점

- 좌표계 = sensor-local (LiDAR 기준). x=forward, y=left, z=up. 전역 회전 불변성 (global rotation invariance) 활용 X (센서 방향에 의미).
- 속도/가속도 미제공 — 11 점 좌표 시퀀스에서 모델이 유도해야 함.
- 외부 데이터 사용 가능, 단 test 데이터 학습 사용 금지.

§2. Dataset — 데이터는 무엇처럼 생겼는가

§2.1 규모와 파일 구조

```
data/
├── train/                # 10,000 개 CSV
│   ├── TRAIN_00001.csv  (header + 11 행)
│   └── ...
├── test/                 # 10,000 개 CSV (라벨 없음)
├── train_labels.csv      # 10,000 행: id, x, y, z (정답)
└── sample_submission.csv # 제출 양식
```

구분	개수	의미
Train	10,000	라벨 있음 (정답 +80 ms 좌표 제공)
Test	10,000	라벨 없음 (예측해서 제출)

§2.2 한 샘플의 모양

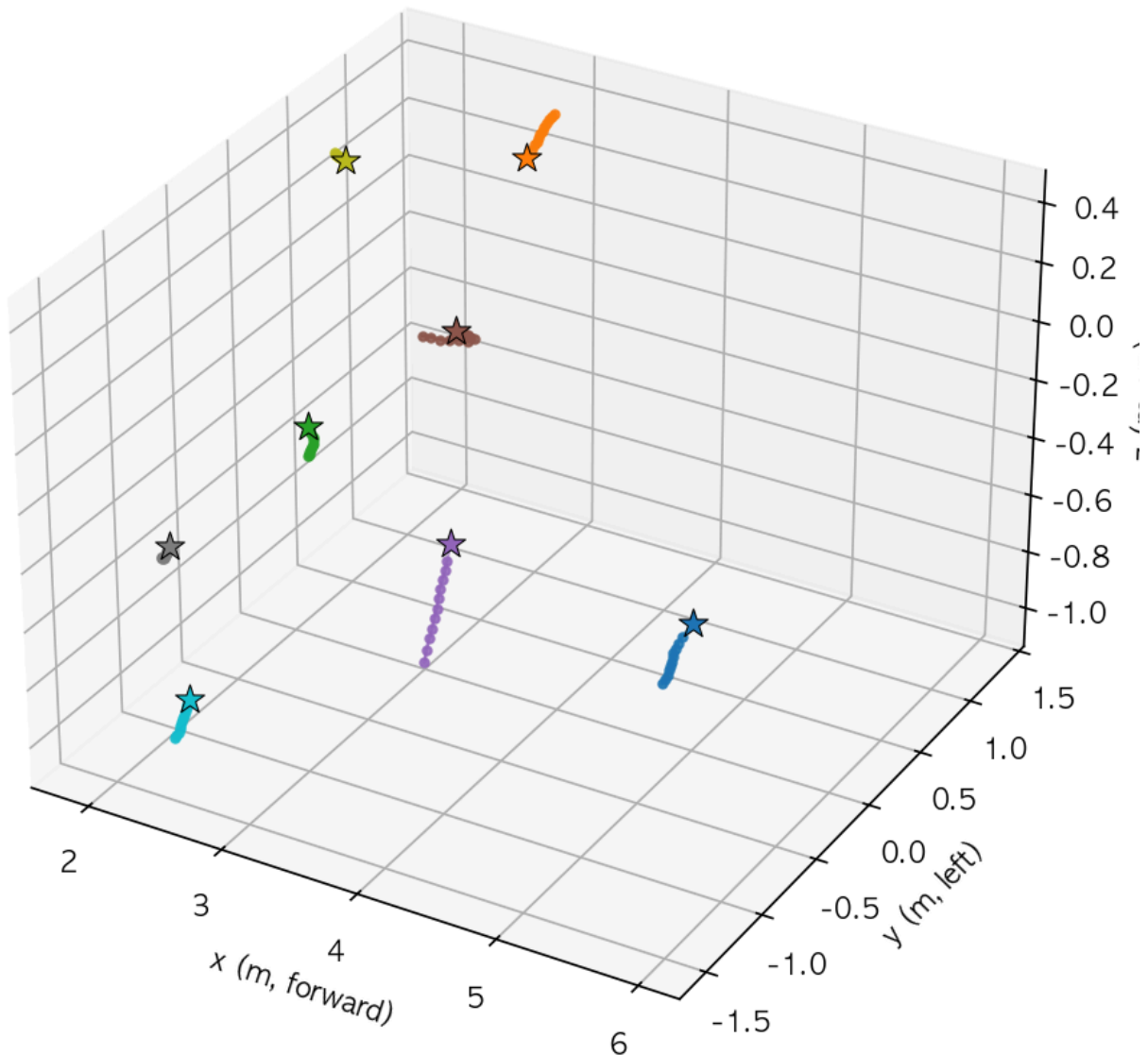
각 trajectory CSV 1 개 = 1 sample. 11 행 × 4 열:

```
timestep_ms, x, y, z
-400, 2.490842, 0.377812, -0.327984
-360, 2.541293, 0.388259, -0.312670
... (총 11 행, 40 ms 간격) ...
0, 2.996920, 0.483173, -0.182941
```

→ 입력은 (11, 3) shape 의 3D 좌표 시퀀스. 정답은 1 개 점 (x, y, z) (+80 ms 시점).

Sample Trajectories (8 random training samples)

● = 관측 11점, ★ = 정답 (+80 ms), 실선 = 관측, 점선 = 마지막 → 정답



- 실선 = 관측된 11 점, 점선 = 마지막 관측 → 정답으로 연결
- 모기 비행은 일반적으로 **연속적이고 부드럽지만**, 일부 sample 에는 급격한 방향 전환 (turn) 이나 가속/감속 (jerk) 이 포함됨

§2.3 No metadata, no scene label

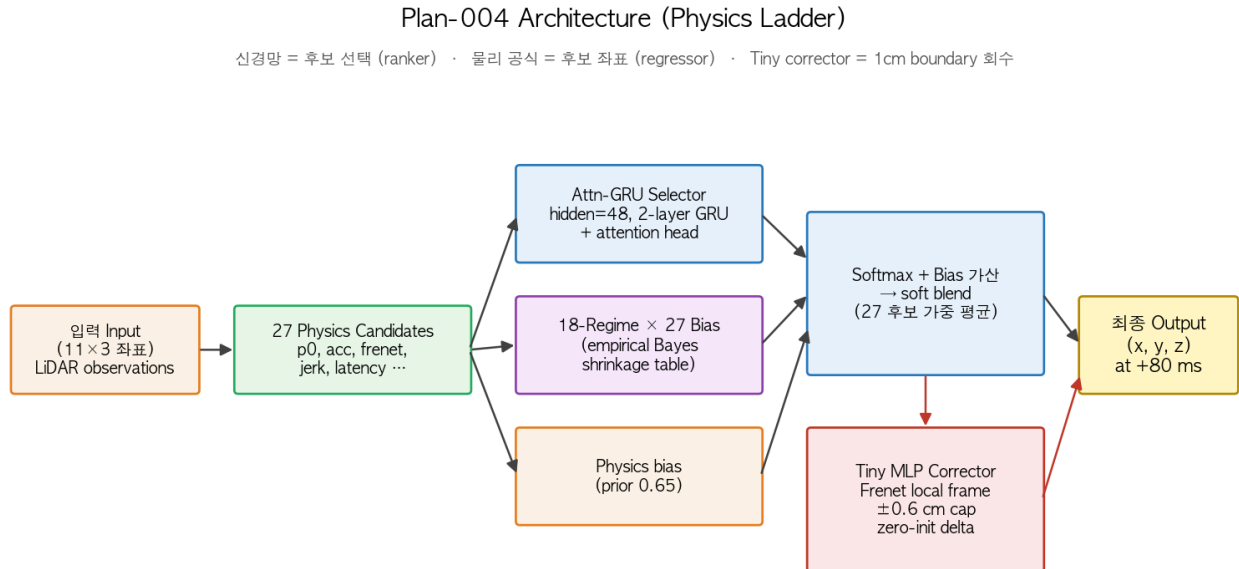
CSV 안에는 좌표만. 속도/가속도/곡률/scene ID 등 모든 **파생 정보**는 모델이 11 점에서 직접 추출해야 한다. 이것이 다음 §3 의 27 *candidates* (= 11 점에서 유도되는 결정적 파생값들) 의 출발점이다.

§2.4 Cross-validation split

- 5-fold CV: `stable_fold_id(sample_id, 5)` (`sample_id` 해시 기반 결정성) — 같은 sample 은 항상 같은 fold.
- 모든 fold 는 train/val 분리 → 5-fold OOF (out-of-fold) 예측을 얻을 수 있다.

§3. Model Architecture — 어떻게 푸는가

§3.1 전체 그림



핵심 흐름:

```
Input (11x3)
↓
27 Physics Candidates      ← 결정적 (deterministic), 신경망 X
↓
Attn-GRU Selector         ← 신경망: 27-way ranking
  ↓ + Physics bias (0.65) + Regime bias (0.45)
27 logits → softmax → soft blend (27 후보 가중 평균)
↓
Tiny MLP Corrector        ← 신경망: Frenet local frame, ±0.6 cm cap
↓
Final (x, y, z) at +80 ms
```

역할 분리 (핵심):

- 신경망 = **ranker / classifier** (후보 선택). 잘하는 일.
- 물리 공식 = **regressor** (절대 좌표 책임). 신경망이 잘 못하는 일.
- Corrector = **boundary 회수기**. 망가뜨릴 수 없도록 *bounded*.

이 분리는 노트북 작성자가 명시적으로 강조한 설계 결정이다 (cell 0 §왜 이런 구조인가):

"이 문제에서 직접 좌표 회귀는 쉽게 노이즈 식별기가 된다. 반대로 후보 물리만 쓰면 *oracle gap* 이 남는다. 그래서 파이프라인은 [물리 후보 → 선택 → 보정] 흐름을 따른다."

§3.2 27 Physics Candidates — Physics Ladder 5 단계

설계자의 narrative 에 따르면 27 개 후보는 다섯 단계 사다리 (Physics Ladder) 로 구성된다. 각 단계는 *가정의 강도* 가 점점 강해진다.

단계 1. 기본 물리 (Baseline) — 5개

가장 보수적. 최근 관측 위치 + 속도/가속도를 약하게 외삽.

candidate	의미
p0_2d1	마지막 위치 + 최근 1-step 이동량 (constant velocity 가정)
acc_2d1_040, _050, _056, _060	최근 속도 + 약한 가속도 (계수 0.4 ~ 0.6)

→ "노이즈가 크면 *과격*한 보정을 막는 anchor" 역할.

단계 2. 개선 물리 — Frenet local frame (6개)

용어 정리: *Frenet local frame* = 월드 좌표축 (x/y/z) 대신 진행 방향 (*tangent*, T) + 수직 회전 방향 (*normal*, N) + 그 외 (*binormal*, B) 의 3축. 자세한 정의는 §3.5 참조.

월드 좌표축 (x, y, z) 대신 **진행 방향** 기준 좌표계로 해석. 비행체의 *local* 운동을 표현.

candidate	의미
frenet_best	최근 진행 방향 + 곡률 기반 대표 후보
frenet_par090_perp000, _par100_perp000	진행 방향 성분 0.9× / 1.0× (속도 유지/약감속)
frenet_par100_perp_neg010, _par090_perp020, _par080_perp020	수직 방향 미세 혼합 (회전/측방 흔들림)

→ "실내/복도/창고 같은 *환경* 을 모델이 직접 맞이지 않아도, *local 물리 방향* 으로 후보를 구성"

단계 3. 강한 물리 — turn / jerk / latency family (16개)

급격한 방향 전환, 가속도 변화, LiDAR 시스템 지연을 후보로 enumerate.

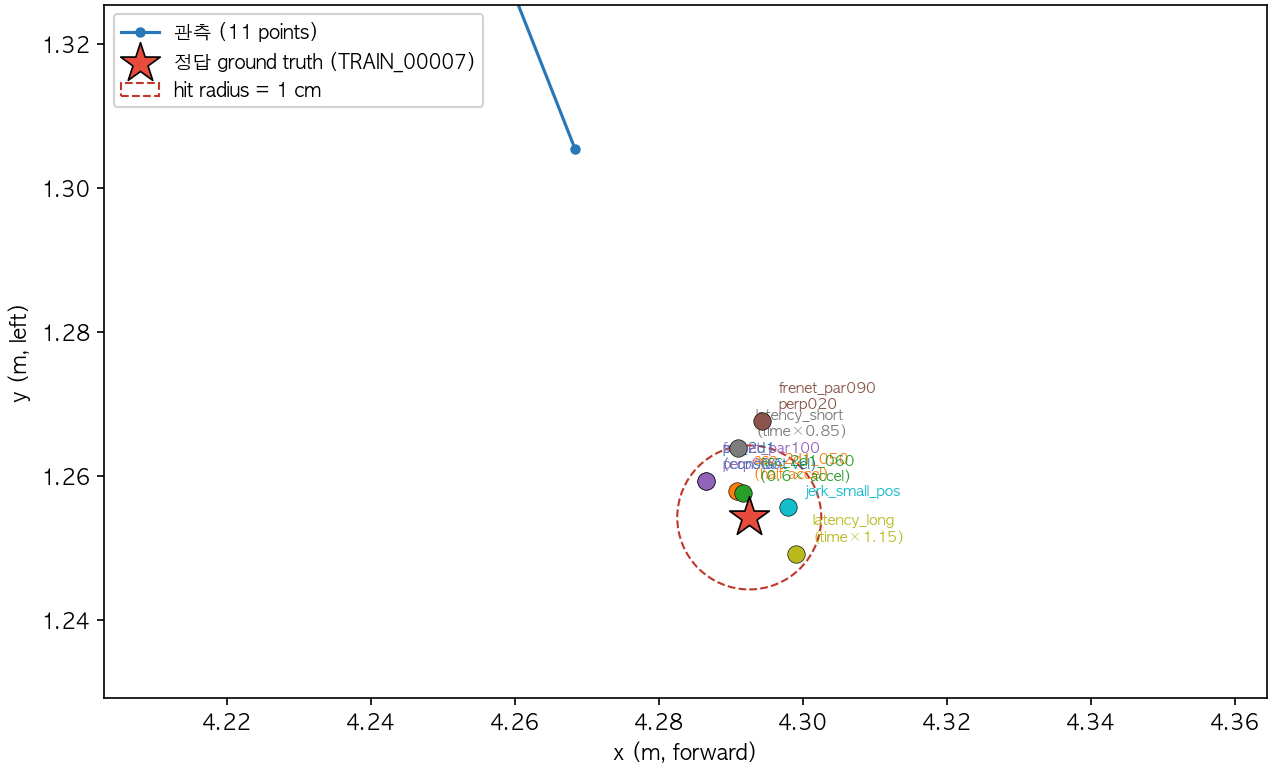
- **Turn family** (4개): frenet_par110_perp_neg020, frenet_par120_perp_neg020, frenet_par120_perp020, frenet_fast_par120_perp_neg020
- **Jerk family** (2개): jerk_small_pos, jerk_small_neg
- **Latency family** (10개): latency_short_frenet_best_085, _092, latency_long_frenet_best_108, _115, latency_long_turn_neg_110, latency_short_turn_pos_090, ... (시간 스케일 0.85 ~ 1.15 변형)

입문자 노트: 개별 candidate 이름을 외울 필요 X. 패턴 = [base family]_[parameter variant] (예: frenet + par100_perp000 = 진행 방향 100% + 수직 0%). family 가 무엇인지 + 27 개로 수가 한정 이라는 점만 기억하면 충분.

Latency 후보의 의도: LiDAR 스캔/추적/좌표 변환의 미세 지연을 *Gaussian noise* 로 평균 보정 하는 대신 *time_scale* 후보로 enumerate 한다. 즉 "지연은 체계적 변형이지 무작위 잡음이 아니다" 라는 도메인 가정.

Cross-ref: 설계자 narrative 의 Physics Ladder 5 단계 = (단계 1·2·3 = 위 후보 family, 본 §) + (단계 4 = Attn-GRU Selector, 다음 §3.3) + (단계 5 = Tiny MLP Corrector, §3.5). 이 문서는 §3.2 ~ §3.5 로 분리 서술.

Figure 4. 같은 입력에 대한 *8개 대표 candidate* 의 분포 (top-down view)
(plan-004의 27개 candidate 중 5개 family 의 대표 — full set 은 regime_distribution.md 참조)



- 위 그림: 같은 입력 trajectory 에 대해 8 개 대표 candidate 가 + 80 ms 시점 어디를 가리키는지
- 빨간 점선 원 = 1 cm hit boundary. 정답이 ★, 후보들은 boundary 안과 밖에 분산
- 27 개 후보 중 어느 하나라도 hit boundary 에 들어가면 selector 의 책임은 "그것을 고르는 것" — 회귀가 아니라 분류 문제

§3.3 Attn-GRU Selector — 27-way ranker

용어 정리 (입문자용): - **GRU** (Gated Recurrent Unit) = 시계열을 한 step 씩 읽으며 hidden state 를 업데이트 하는 신경망 (LSTM 의 경량 버전). - **logit** = softmax 직전의 raw score. 음수/양수 모두 가능. 큰 값일수록 "더 그럴듯한 후보". - **softmax** = N 개 logit 을 확률 분포 (합 = 1) 로 변환하는 함수. $\text{softmax}(x_i) = e^{x_i} / \sum_j e^{x_j}$. - **attention** = "시퀀스의 어느 step 이 중요한가" 를 학습된 weight 로 결정하는 layer. 모든 step 을 균등 평균 하는 대신 중요한 step 에 더 큰 weight.

구조 (src/pb_0_6822/selector.py L697-726 CandidateAttentionGRUSelector):

```

sequence features (9D: speed, accel_components, jerk, curvature, dir_flag)
↓
2-layer GRU (hidden=48)
↓
attention head (query/key over sequence)
↓
context vector
↓
concatenate with candidate features (20D per candidate)
↓
linear → 27 logits

```

Inference 시 logit 조정 (3 항 가산):

$$\text{score}_c = \text{logit}_c + \alpha \cdot b_c^{\text{phys}} + \beta \cdot b_{r,c}^{\text{regime}}$$

- $\alpha = 0.65$ (physics prior strength)
- $\beta = 0.45$ (regime prior strength)
- b_c^{phys} = 후보 family 별 사전 확률 (예: `frenet_best` 가 기본적으로 잘 맞으므로 약간 boost)
- $b_{r,c}^{\text{regime}}$ = 다음 §3.4 의 18 × 27 표에서 추출

설계 의도 (cell 0 §4): "GRU 는 최근 상태를 안정적으로 요약. BiGRU 보다 단방향인 노이즈 흡수 덜 함. Attention 은 과거 사건 외우기가 아니라 후보 점수 차이를 부드럽게 만드는 장치."

§3.4 18-Regime × 27-Candidate Empirical Bayes 표

Regime 정의 (3-축 quantile binning):

축	bin 경계	bin 수
speed	[0.0176, 0.0290]	3
curvature	[0.0874, 0.1923]	3
speed_slope (가속/감속)	[0.0108]	2

→ `regime = speed_bin × 6 + curve_bin × 2 + slope_bin` ∈ {0, ..., 17}.

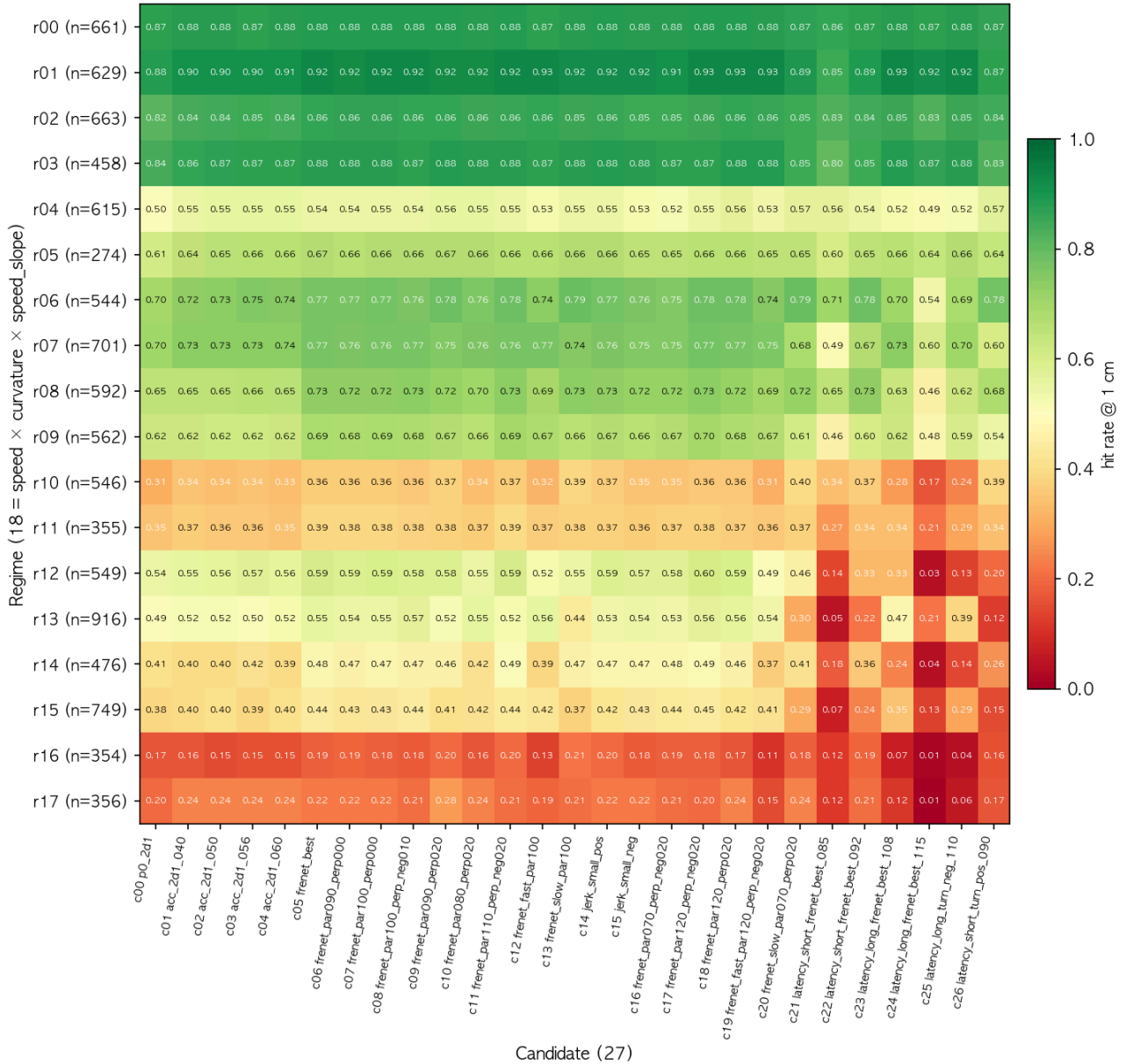
입문자 노트: 위 인덱싱 공식 자체는 신경 안 써도 됨. "18 = 3 × 3 × 2 (속도 3 단계 × 곡률 3 단계 × 가/감속 2 단계) 의 격자" 정도만 기억. 각 sample 이 18 칸 중 한 칸에 들어간다는 것이 핵심.

각 (regime, candidate) cell 에 **train OOF hit rate** 를 empirical Bayes shrinkage 로 안정화 (prior strength = 18 sample, `candidate_regime_bias`, `src/pb_0_6822/selector.py` L380-403):

$$\hat{p}_{r,c}^{\text{EB}} = \frac{n_{r,c} \cdot \hat{p}_{r,c} + \pi \cdot \hat{p}_{\text{global}}}{n_{r,c} + \pi}, \quad \pi = 18$$

표 자체 시각화:

Figure 5. 18 regime $\times$ 27 candidate hit-rate (@ 1 cm) — 실측 train OOF
 녹색=잘 맞춤, 빨강=거의 못 맞춤. selector logit 에 이 표 기반 regime_bias(0.45) 가산.



읽는 법: - 세로축 (y): 18 개 regime. 옆에 sample 수 (n=...) 표기 — 최소 274, 최대 916 (모두 50 이상 → shrinkage 신뢰성 영역). - 가로축 (x): 27 candidate. c00 ~ c26. - 색: 녹색 = 잘 맞춤, 빨강 = 못 맞춤. - 예: regime 0 (저속/직진/감속) → c00 ~ c20 대부분이 0.85+ (밝은 녹색). 반면 regime 16 (고속/고곡률/가속) → 전 candidate 가 0.15 ~ 0.20 (어두운 빨강). - 일부 cell 은 *hyper-specialized* — 같은 regime 안에서 특정 candidate 만 유난히 잘 맞거나 (ratio ≥ 1.5) 못 맞춤 (ratio ≤ 0.5). plan-004 결과: 19개 *hyper-specialized cell* 확인됨 (대부분 latency family 가 regime 12 ~ 17 에서 *under-perform* — 후속 plan 의 regime 재설계 anchor 정보).

→ Inference 시: 입력 trajectory 의 regime r 을 계산한 뒤, 해당 행의 27 개 hit-rate 를 logit 에 0.45 가산. 즉 "이 regime 에서 잘 맞는 candidate 에 사전 우대".

§3.5 Tiny MLP Corrector — boundary 회수기

용어 정리 (입문자용): - **MLP** (Multi-Layer Perceptron) = 가장 기본적인 fully-connected 신경망 (입력 → 은닉 층 → 출력 의 단순 layer stack). - **LayerNorm** = 입력 feature 의 평균/분산을 정규화해 학습을 안정화하는 layer. - **residual block** = $output = f(x) + x$ 형태로 입력 신호를 보존하며 변환을 더하는 구조. 깊은 신경망 학습의 핵심 trick.

구조 (src/pb_0_6822/boundary.py L166-196 TinyCorrectionNet):

```
20D context (속도, 가속도 metric, 후보 confidence)
↓
LayerNorm
↓
ResidualMLP block × 2 (hidden=64)
↓
delta head → 3D residual (Frenet local frame)
    ★ zero-init: 학습 시작 시 보정량 = 0
↓
cap ±0.006 m (= 0.6 cm)
↓
local → world 변환 → selector soft blend 좌표에 더함
```

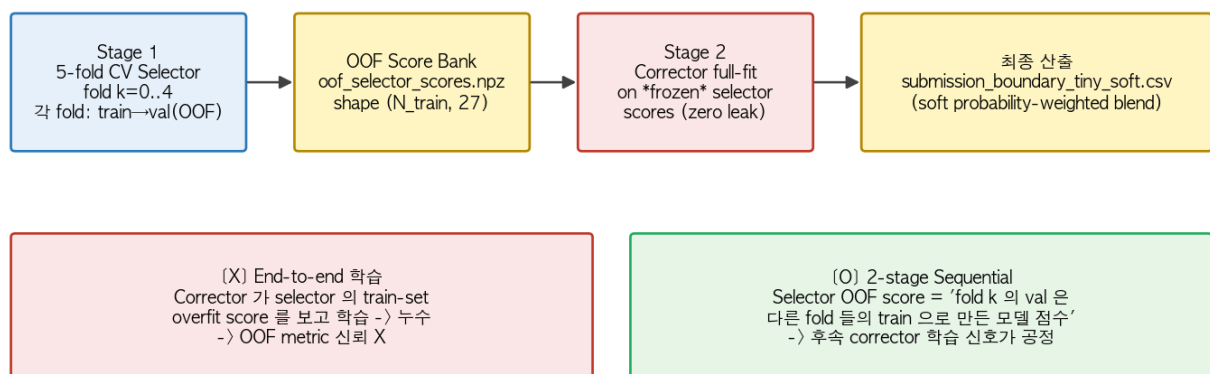
Frenet local frame: world 좌표 (x, y, z) 대신 **tangent** (진행 방향) / **normal** (수직 회전 방향) / **binormal** (이외) 의 3축. local_frame() 함수 (src/pb_0_6822/boundary.py L35-50).

Boundary weighting: 학습 시 sample 마다 loss 가중치 조정. - 오차 << 1 cm 인 sample (이미 hit) → 낮은 가중치 - 오차 >> 1 cm 인 sample (회복 불가) → 낮은 가중치 - 오차 ≈ 1 ~ 2 cm 인 sample (**flip 가능 zone**) → 가장 높은 가중치

→ "학습 자원이 cap 범위 안에서 hit flip 가능 한 sample 에만 흐르도록" (cell 0 부록 §A).

§3.6 2-stage Sequential 학습

2-stage Sequential Training (누수 차단을 위한 OOF interface)



Stage	무엇	산출
1	5-fold CV Selector. fold k 학습 → fold k 의 val 예측 (OOF)	<code>oof_selector_scores.npz</code> shape (10000, 27)
2	Corrector full-fit. 입력 = Stage 1 의 OOF score bank. selector 는 <i>frozen</i> .	<code>submission_boundary_tiny_soft.csv</code>

왜 **sequential 인가** (cell 0 §5): end-to-end 학습 시 corrector 가 selector 의 *train-set overfit score* 를 보고 학습 → 누수. OOF score bank 로 *frozen interface* 를 만들면 corrector 학습 신호가 공정해진다.

§4. Why This Architecture is Justified — 정당화

이 §은 위 §3 의 각 설계 결정이 왜/정당한가를 메트릭 → 설계로 가는 인과 사슬로 풀어쓴다. 설계자 (노트북 작성자) 의 의도 를 그대로 인용/확장한다.

§4.0 설계자의 핵심 원칙 (한 문장)

"평균을 최소화하지 말고, 평가 지표가 실제로 보상하는 좁은 영역에 자원을 집중하라." — notes/PB_0.6822 코드공유.ipynb cell 0 부록 §A·B 공통 정신

이 문장이 §4.1 ~ §4.6 의 모든 정당화의 공통 뿌리다. 평가 메트릭이 hit@1cm 이라는 임계값 비율이므로, 평균 거리 최소화 (L2/MSE) 대신 (a) 평가가 보상하는 영역 (1cm 안) 에 자원 집중 + (b) 평가와 무관한 영역 (5cm 밖) 에 자원 낭비 X 라는 비대칭 자원 배분을 모델 구조 곳곳에 박제한다.

§4.1 메트릭이 회귀를 분류로 만든다

관찰: hit@1cm 은 임계값 메트릭. 1 cm 안에 들면 +1, 밖이면 0. 평균 거리 0.9 cm 인 분포가 좁은 예측 < 평균 0.5 cm 인 분포가 넓은 예측 일 수 있다 (Figure 7).

결론: 회귀 모델이 평균 거리를 줄이는 방향으로 학습되어도 hit rate 는 직접 보상되지 않는다. 대신:

1. 예측 공간을 물리적으로 가능한 discrete 후보 집합으로 좁힌다 → 27 candidates.
2. 신경망의 역할 = 후보 선택 (분류) → Attn-GRU Selector.
3. 분류는 회귀와 달리 후보별 hit/miss 확률을 직접 학습 가능 → 메트릭과 학습 목표 정렬.

실증: plan-001 baseline (회귀, polyfit) = LB 0.60. plan-004 (분류 기반) = LB 0.6806. +0.08 absolute lift.

§4.2 왜 신경망에 절대 좌표를 시키지 않는가

설계자 인용 (cell 0 §왜 이런 구조인가):

"이 문제에서 직접 좌표 회귀는 쉽게 노이즈 식별기가 된다."

의미: - 회귀 모델은 입력의 sub-cm 변동까지 출력에 반영하려 함 → 결국 입력 노이즈를 학습 해 generalization 깨짐.
- 분류 모델은 클래스 (= 후보) 간 boundary 만 학습 → 입력 노이즈에 robust.

역할 분리의 이득:

책임	담당	이유
절대 좌표 (mm 단위)	27 물리 공식	결정적, 노이즈 무관
후보 ranking	Attn-GRU	신경망이 잘하는 일
미세 보정 (< 0.6 cm)	Tiny Corrector	bounded → 망가뜨릴 수 없음

→ 신경망은 자기가 잘하는 일만 한다.

§4.3 18-regime bias 의 정당화

Why regime? — 같은 후보라도 regime 에 따라 정확도가 다르다.

Figure 5 의 표 관찰: - regime 0 (저속/직진/감속): 모든 후보가 0.85+ → 거의 균등 - regime 16 (고속/고곡률/가속): 후보별 0.01 ~ 0.20 으로 큰 분산 → candidate 선택이 중요

→ "regime 별 후보 우선순위 (prior)" 가 의미 있다.

Why empirical Bayes shrinkage? — regime 별 sample 수가 다르다 (min 274 ~ max 916). cell 별 hit-rate raw 추정은 sample 적은 cell 에서 noisy. **shrinkage = noisy cell 을 global mean 쪽으로 끌어당김.**

$$\hat{p}_{r,c}^{\text{EB}} = \frac{n_{r,c} \cdot \hat{p}_{r,c} + \pi \cdot \hat{p}_{\text{global}}}{n_{r,c} + \pi}$$

- $n_{r,c}$ 가 크면 → raw 추정 신뢰 → shrunken $\approx$ raw.
- $n_{r,c}$ 가 작으면 → shrunken $\approx$ global mean (regularized).

실증: plan-004 의 18 regime 모두 sample ≥ 50 (min 274) → shrinkage prior ($\pi = 18$) 가 충분히 작아 raw 정보 보존 + noise 만 안정화. `degenerate_regimes = []` (regime_distribution.json).

§4.4 Corrector 의 안전성 — 왜 망가뜨릴 수 없는가

설계자 인용 (cell 0 부록 §A):

"Tiny correction 은 세 trick 이 아니라 한 원칙의 세 면"

요소	역할
cap = 0.006 m	보정 범위 = 최대 0.6 cm. 5 cm 빗나간 sample 은 회복 불가능
Boundary weighting	학습 자원이 cap 범위 안에서 hit flip 가능 한 sample (≈ 1 cm 부근) 에만 흐름
Zero-init delta head	학습 시작 시 보정량 = 0. 첫 epoch 부터 후보를 망가뜨리지 않음

공통 원리: corrector 의 좁은 능력 범위 안에서만 학습 신호를 흘리고, 나머지는 건드리지 않는다.

→ Selector(큰 결정)와 Corrector(boundary flip)의 분업이 loss weighting 으로 명시 되어 있다 (selector 가 5 cm 실패한 sample → corrector 가 회수 시도 X, 학습 가중치 0).

실증: plan-004 결과 (results.md §2.2): - Selector OOF soft hit = 0.6624 - Corrector OOF soft hit = 0.6718 (+0.0094 lift, 망가뜨리지 않으면서 회수) - `corrector_no_convergence` severe trigger 미발생 (0.6718 > 0.6624 보장)

§4.5 Latency family — hypothesis enumeration, *not* noise filtering

맥락: **Kalman filter** = 센서 노이즈를 Gaussian (정규분포) 로 가정 하고 매 step 추정값을 통계적 평균 으로 보정 하는 고전 시계열 추정 기법. "자연 = 평균을 추정해야 할 무작위 잡음" 이라는 가정에서 있다. 이 모델은 그 가정 자체를 뒤집는다.

설계자 인용 (cell 0 부록 §B):

"기존 (Kalman 등): 센서 지연을 *Gaussian noise* 로 보고 통계적 평균 보정. 이 방법: 지연을 $time_scale \in \{0.85, 0.92, 1.08, 1.15\}$ 후보로 *enumerate* → 분류기로 선택."

도메인 가정의 차이:

접근	가정	결과
Noise filter (Kalman)	지연은 무작위 잡음	평균 추정 $X \rightarrow$ 불확실성으로 흡수
Hypothesis enumeration	지연은 체계적 변형 (몇 가지 모드)	모드별 후보 $\rightarrow$ selector 가 어느 모드 인지 결정

이는 §4.1 의 원칙 ("평균 최소화 $X \rightarrow$ 메트릭이 보상하는 영역에 자원 집중") 의 **latency 영역 적용** 이다. 같은 정신:

- Latency family: 평균 지연 추정 $X \rightarrow$ 지연 시나리오 *enumerate* → 정답 시나리오 선택
- Boundary correction: 평균 거리 최소화 $X \rightarrow$ flip 가능 sample 가중치 집중

→ §A.B 의 공통 정신 = **통계적 평균 최소화 $\rightarrow$ 메트릭의 결정적 영역에 자원 집중.**

§4.6 2-stage Sequential 의 정당화

Why not end-to-end? — corrector 가 selector 의 *train-set overfit score* 를 보고 학습하면, OOF 성과와 train 성능이 따로 놀게 됨 $\rightarrow$ 누수.

Why OOF score bank? — fold k 의 val partition 에 대한 selector 점수는 *다른 fold 들의 train 으로 학습된* 모델의 출력 $\rightarrow$ train data 미노출. Corrector 가 이 OOF score 를 입력으로 학습하면, train 성과와 OOF 성과가 일관됨.

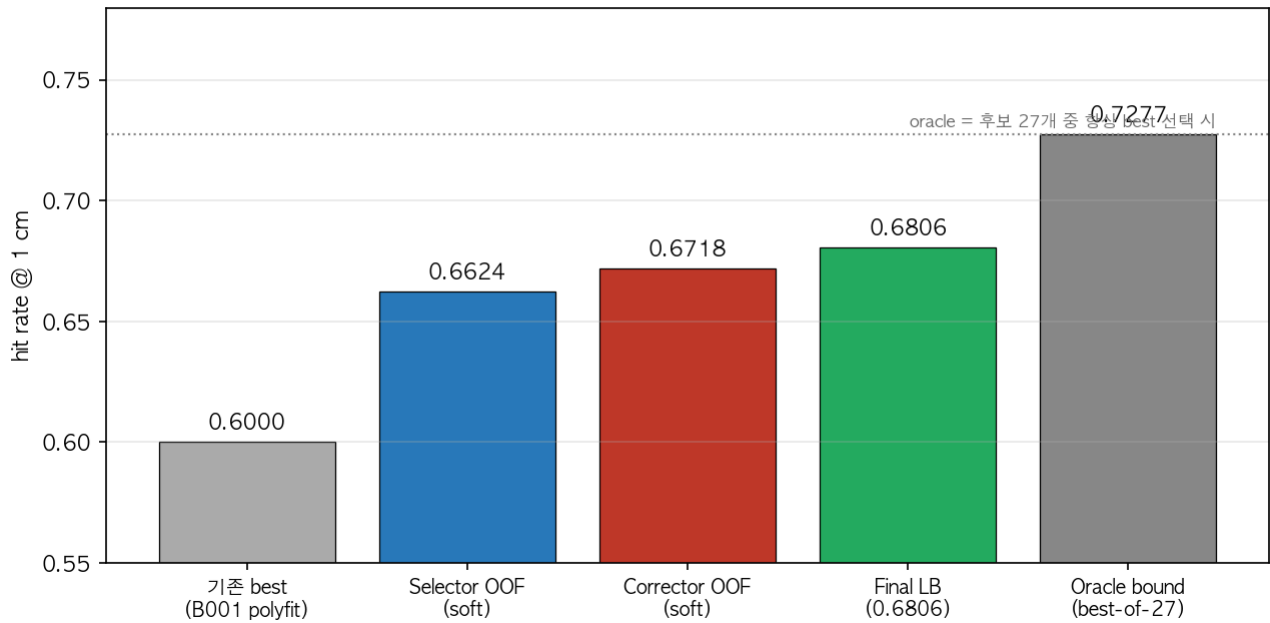
부수 이득: Selector 와 Corrector 의 모듈러 ablation 이 자유로워짐. Selector 만 교체, Corrector 만 교체 가능.

§4.7 Oracle bound 와 남은 헤드룸 — 어디까지 갈 수 있는가

단계별 hit rate 진화:

단계	hit@1cm	누적 lift
기존 best (B001 polyfit, plan-001)	0.6000	— (baseline)
Selector OOF (plan-004 stage 1)	0.6624	+0.0624
Corrector OOF (plan-004 stage 2)	0.6718	+0.0718
Final LB (plan-004 제출)	0.6806	+0.0806
Oracle bound (best-of-27)	0.7277	(ceiling)

Figure 8. plan-004 가 만든 lift 의 분해
(0.60 → 0.6806 = +0.0806; 남은 oracle 헤드룸 = 0.0471)



Oracle 의 의미: 만약 selector 가 **항상 정답 후보를 선택** 한다면 hit rate = 0.7277. 즉 **27 candidate 자체로 70%+의 문제는 원리적으로 풀린다**. 남은 30% 는 후보 자체로 cover 안 됨 (후보 family 확장 필요).

현재 헤드룸: $0.7277 - 0.6806 = 0.0471$ (4.7pp). 이 중 절반은 selector ranker 개선, 절반은 candidate 자체 확장으로 회수 가능 (plan-004 §5 다음 plan 후보 참조).

§5. Limitations & Caveats

1. **노트북 0.6822 vs 본 구현 0.6806 — 재현 X:** 노트북 작성자의 환경/seed/data split 이 미세하게 다를 수 있어 0.0016 gap 발생. plan-004 의 목표는 *재현* 이 아니라 *우리 데이터에 적용 시 점수 측정*이었음 (cell 0 narrative 의 두 갈래 중 첫 번째).
2. **Boundary corrector OOF 가 fold 0 val 한정:** 본 구현은 5-fold concatenated OOF 미박제 (`overall_oof_hit_soft` 키 미생성). 후속 plan 에서 5-fold corrector 모드 활성화 시 `corrector_no_convergence` 자동 판정 logic 도 같이 활성화.
3. **R_HIT = 1 cm 이 dacon 측 메트릭과 정확히 일치한다는 가정:** 대회 page 가 "레이저 유효 반경" 이라는 표현만 명시. 실측 LB 0.6806 가 우리 hit-rate 계산과 일관되므로 사실상 확인됨.
4. **선형/완만 trajectory 가 80% 이상:** regime 0-9 에 sample 의 약 60% 가 분포 → "어려운 sample" 은 regime 10-17 에 집중. 즉 본 모델은 *쉬운 segment* 에서 0.85+ hit, *어려운 segment* 에서 0.15 ~ 0.40 hit 의 극화된 성능.
5. **27 candidate 는 도메인 별 설계:** 다른 비행체 (드론, 새 등) 에 동일 framework 적용 시 candidate family 재설계 필요. *원리 (Physics Ladder)* 는 일반적이지만 *구체 candidate* 는 모기 비행에 tuned.

§6. References

카테고리	파일
원본 노트북 (설계자 narrative)	notes/PB_0.6822 코드공유.ipynb cell 0 "Algorithm Notes: Physics Ladder"
대회 개요	notes/competition-overview.md
Plan 본문 (autonomous loop spec)	plans/plan-004-pb-0-6822-fullrun.md
Plan 결과 (실측 metric)	plans/plan-004-pb-0-6822-fullrun.results.md
18×27 regime 분석	analysis/plan-004/regime_distribution.{json,md}
LB 제출 로그	analysis/plan-004/lb_log.md
Selector 구현	src/pb_0_6822/selector.py (L697-726 Attn-GRU class, L380-403 empirical Bayes)
Corrector 구현	src/pb_0_6822/boundary.py (L166-196 TinyCorrectionNet, L35-50 Frenet)
Orchestrator	src/pb_0_6822/run_full.py

Appendix A. 자주 묻는 질문 (FAQ)

Q1. 왜 27 candidate 인가? 더 많으면 안 되나? A. 후보를 늘리면 oracle (best-of-N) 은 올라가지만 selector 가 노이즈까지 배울 위험도 같이 커진다 (cell 0 §3 인용: "후보 family 가 넓어질수록 candidate_oracle 과 실제 selector hit 사이 gap 이 생긴다"). 27 은 oracle 0.7277 와 selector lift 사이의 경험적 sweet spot. 후속 plan 에서 ablation 으로 24/30/36 비교 가능.

Q2. Frenet frame 이 왜 world 좌표보다 좋은가? A. 같은 물리적 보장 (예: 측방 0.5 cm 흔들림) 이 world 좌표에서는 방향에 따라 (x/y/z 각기) 다르게 표현되지만, Frenet 에서는 진행 방향 기준으로 일관 표현된다. 결과적으로 corrector 가 적은 sample 로 일반화 가능 (학습 효율↑).

Q3. Empirical Bayes shrinkage 의 prior strength = 18 은 어떻게 정해졌나? A. 노트북 default. 직관: 18 = "각 cell 이 추가 정보 없이 18 개 sample 만큼의 prior 정보를 가진다" 의미. min cell sample 이 274 이므로 shrinkage 강도 = $18 / (274+18) \approx 6\%$ (= raw 추정 94% 신뢰). 후속 plan 에서 5/10/30 ablation 가능.

Q4. 왜 cap = 0.6 cm (= 0.006 m) 인가? A. (a) hit boundary 가 1 cm 이므로 cap 이 이보다 작아야 boundary flip 만 가능 (1 cm 밖 sample 회복은 selector 책임). (b) LiDAR 센서 노이즈 $\sigma \approx 0.5 \sim 0.7$ cm 와 같은 scale → corrector 가 센서 노이즈를 학습하는 것을 막음.

Q5. 그럼 이 모델의 weakness 는? A. (a) Regime 14-17 (고속/고곡률) 의 hit rate 가 0.15 ~ 0.40 으로 매우 낮음 — 27 후보 자체가 이 영역을 cover 못함. (b) Fold 0 val 한정 corrector 평가 → 5-fold concatenated OOF 미박제. (c) End-to-end seed ensemble / multi-corrector blend 미시도 (후속 plan).

문서 작성: 2026-05-16. plan-004 (PB_0.6822 fullrun) LB 0.6806 확정 후 외부 공유용 입문자 해설.